SHIFTLEDGER

WEB EXPLOITATION CHALLENGE- OFFENSIVE WRITEUP

Target	ShiftLedger (Meridian Logistics internal portal)
Assessment Type	Authenticated web application exploitation, single target machine
Domain	Web Exploitation
Operating System	Ubuntu 24.04 LTS
Prepared By	Mukund Ugale
Date	18-10-2026

Table of Contents

No.	Contents
1.	Executive Summary
2.	Introduction
3.	Scope Of Assessment
4.	Starting Access And Rules Of Engagement
5.	Testing Methodology
6.	Tools Used
7.	Milestones
8.	Reconnaissance
9.	Stage 1 - Mass Assignment Self-Approval
10.	Stage 2 - Trusted Field Role Escalation
11.	Stage 3 - Constrained Server-Side Template Injection
12.	Stage 4 - Insecure Deserialization
13.	Final Proof Of Compromise
14.	Recommendations
15.	Conclusion
16.	References

1. Executive Summary :

This report documents an authenticated web application exploitation exercise performed against ShiftLedger, a purpose-built single-machine challenge developed for this assessment. Starting from a single low-privilege employee account, four chained vulnerabilities were identified and successfully exploited, resulting in full compromise of the target machine (root access). Each finding is documented below with technical evidence, impact analysis, and remediation guidance.

All four vulnerabilities were independently designed and implemented for this challenge and do not correspond to any default-credential, textbook-injection, or single-exploit shortcut; each stage required genuine analysis to identify and exploit, as detailed in the per-stage sections below.

2. Introduction :

ShiftLedger is the internal payroll and shift-scheduling portal for a fictional logistics company, Meridian Logistics. Warehouse staff log shifts, clock in and out, and submit payroll adjustment requests. Supervisors review and approve these requests within their own department, and the Finance team exports approved adjustments as payslip batches via a server-rendered memo feature. A separate backend service, running independently of the web application, periodically processes finance export jobs and writes them to a protected archive directory.

This assessment was conducted to determine how far an attacker could escalate privileges starting from nothing more than a single employee-level web application account.

3. Scope Of Assessment :

Target Application	ShiftLedger (single Ubuntu 24.04 machine)
Assessment Type	Authenticated web application exploitation
Testing Environment	Local lab VM, deployed via the provided setup.sh
In-Scope	Authentication, authorization, session management, server-side template rendering, background service input handling
Out-of-Scope	Network infrastructure, denial-of-service testing, third-party services

4. Starting Access And Rules Of Engagement :

Target	http://<target-ip>/
Starting Account	alice / Welcome123! (role: employee)
Authorization	Challenge machine deployed specifically for this assessment
Objective	Full compromise of the target machine (root access)

No credential discovery, brute-forcing, or password guessing was required or attempted; the starting account above was provided as the defined entry point for the assessment.

5. Testing Methodology :

Manual testing techniques were used throughout, in line with standard web application testing practice (informed by the general approach of the OWASP Web Security Testing Guide). HTTP requests and responses were analyzed and modified using Burp Suite, and each finding was manually verified and chained into the next stage rather than treated in isolation.

Testing activities:

- Information gathering and authenticated endpoint enumeration
- Authorization and access-control testing
- Input validation and server-side template injection testing
- Analysis of backend service input handling
- Chained exploitation and verification of each milestone
- Documentation of findings with reproducible evidence

6. Tools Used :

Burp Suite	Intercepting, modifying, and replaying HTTP requests (Repeater)
ffuf	Authenticated endpoint enumeration
Firefox / Chrome	Accessing and testing the web application
Browser DevTools	Inspecting responses and session state
Python 3	Constructing the Stage 4 deserialization payload

7. Milestones :

Stage	Milestone	Mechanism
0	Recon	Authenticated endpoint enumeration
1	employee -> supervisor	Mass assignment on the amend endpoint
2	supervisor -> finance_admin	Trusted client-supplied role field
3	finance_admin -> code execution (svc_shiftledger)	Constrained SSTI, filter bypass required
4	svc_shiftledger -> root	Insecure deserialization in a root daemon

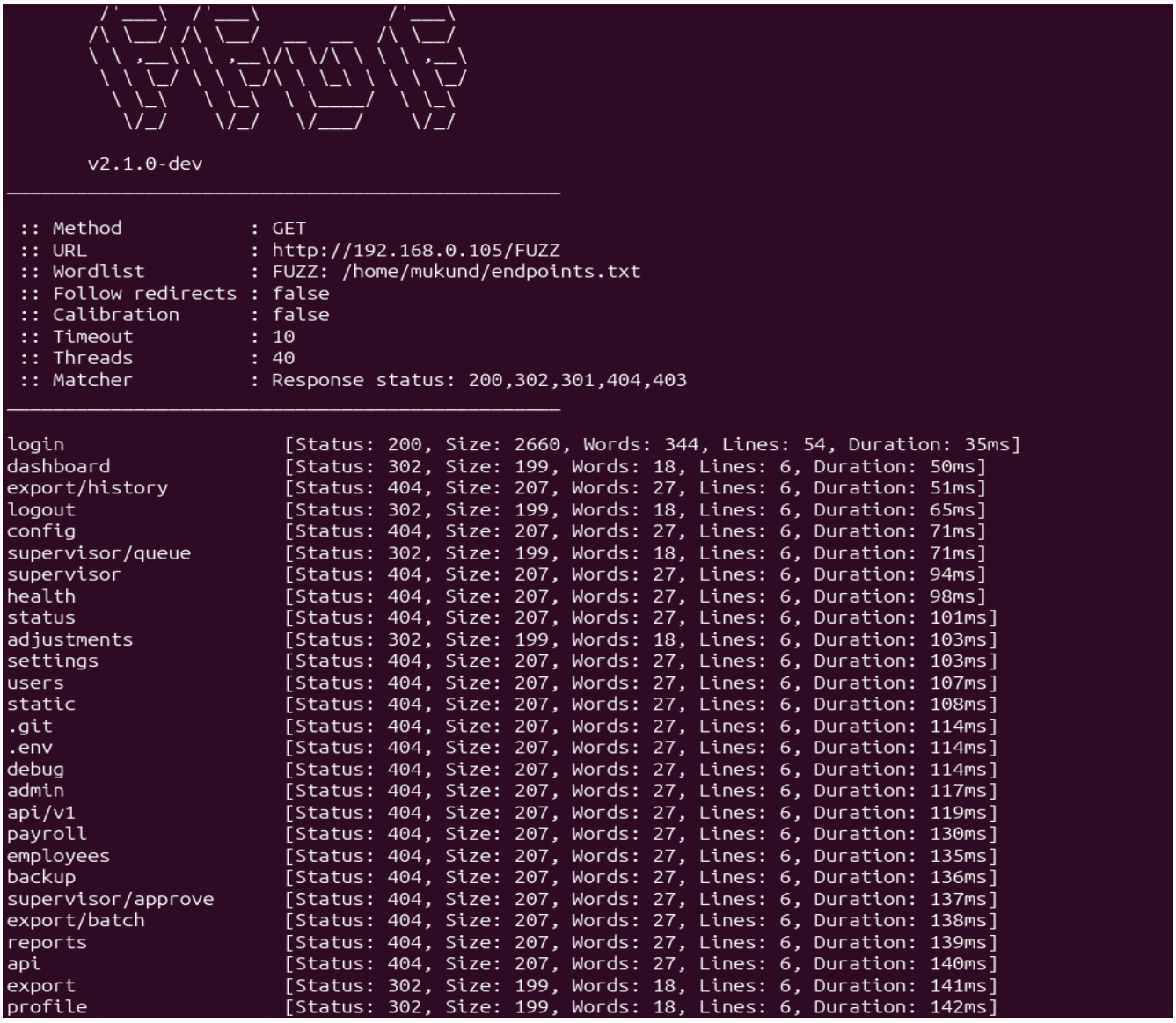
8. Reconnaissance :

With the provided employee credentials, the application was explored as a normal user: shift calendar, clock-in, and the 'Request Adjustment' workflow. Authenticated endpoint enumeration ffuf was used to map routes beyond what the UI links to directly, surfacing /supervisor/queue as a route that exists but returns 403 for an employee session, indicating a role-gated feature worth further investigation.

Command :

ffuf -u http://192.168.0.105/FUZZ -w endpoints.txt -mc 200,302,301,404,403

Output :



9. Stage 1 — Mass Assignment Self-Approval

Employee —> Supervisor

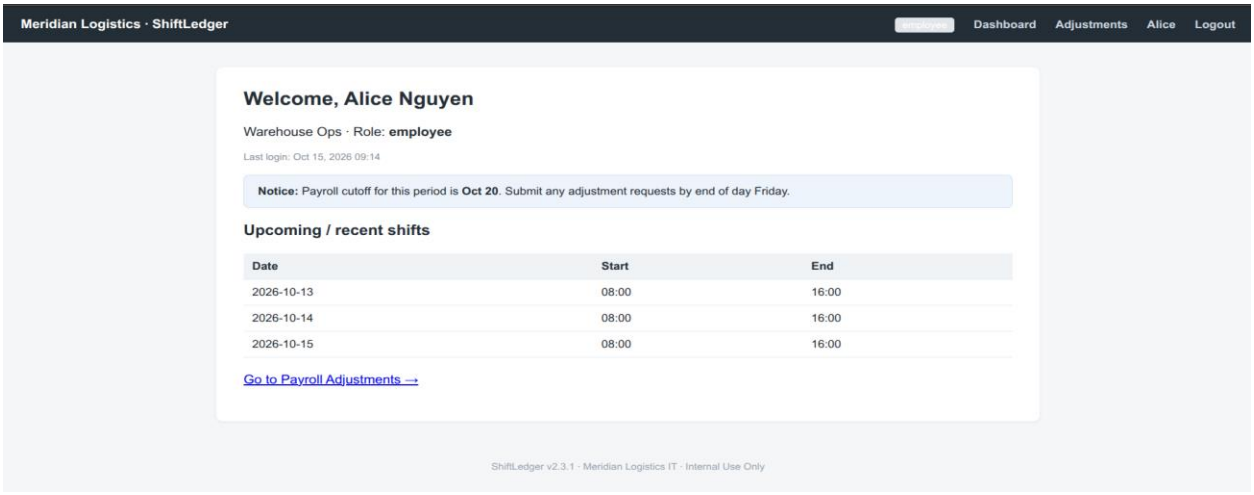
Severity	High
OWASP Category	A01:2021 – Broken Access Control
CWE	CWE-915 – Improperly Controlled Modification of Dynamically-Determined Object Attributes
Affected Endpoint	POST /adjustments/<id>/amend

Description:

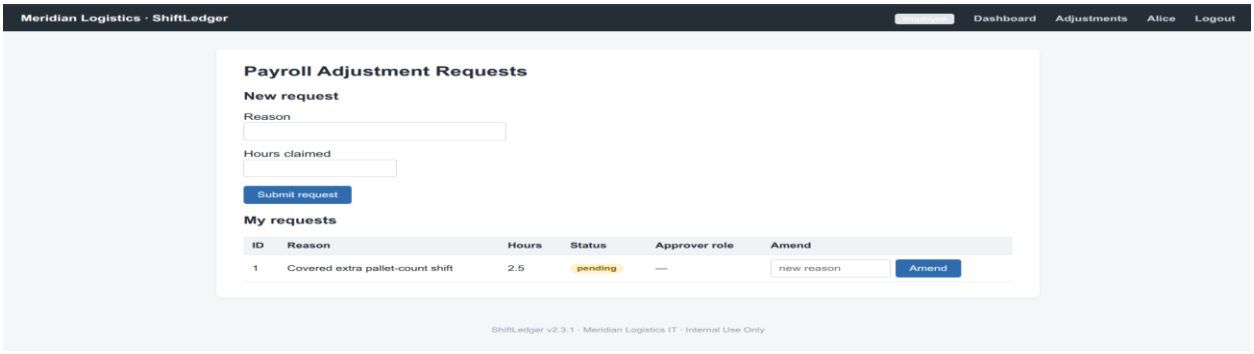
The Adjustments page exposes an 'Amend' action intended only to let an employee correct the reason text on their own pending request. Inspecting the request in Burp shows the UI form submits only a reason field. Testing whether the server accepts additional, undocumented fields revealed that status and approver_role are also accepted and applied with no field-level restriction. This is distinct from an Insecure Direct Object Reference: no object-ID guessing or cross-user access is involved — the attacker only ever modifies their own request object, via fields the UI never intended to expose.

Steps to reproduce :

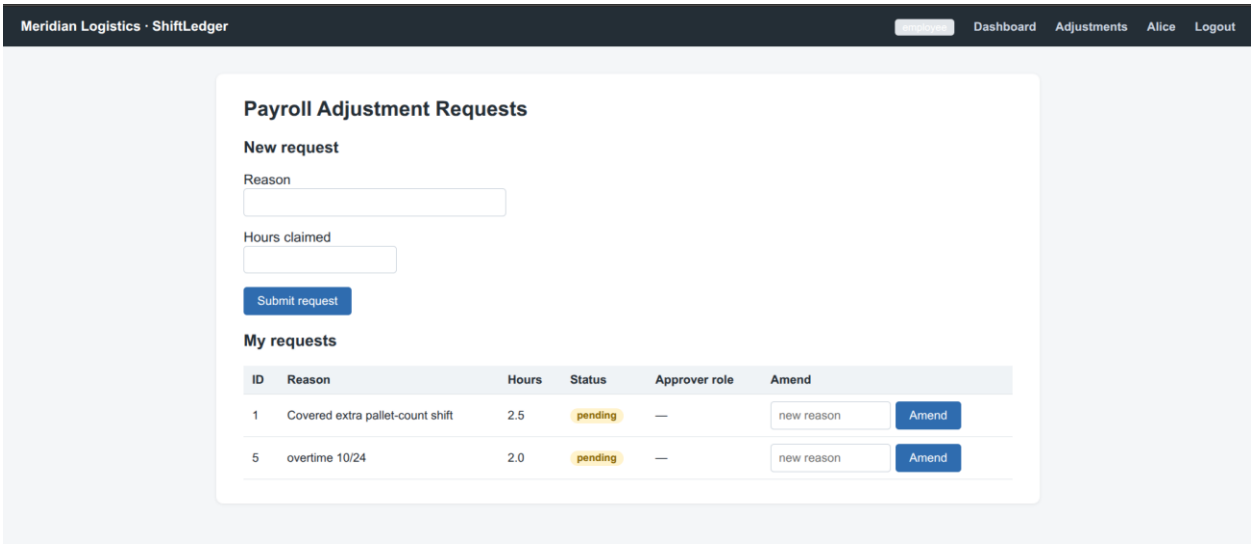
1. Go to <target-ip> and login with provided credentials : alice / Welcome123!



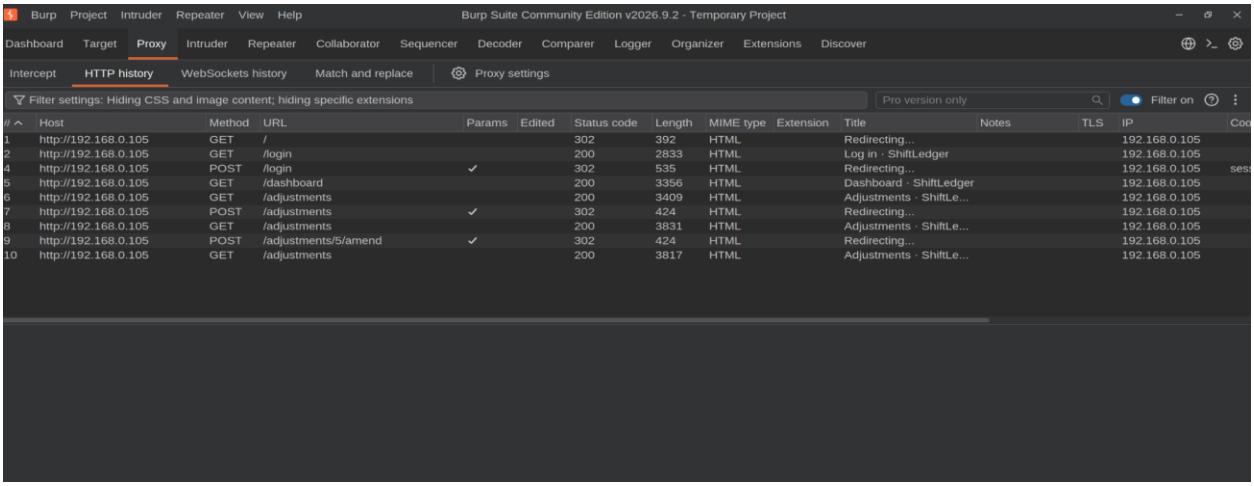
2. Go to Adjustments/Go to payroll Adjustments



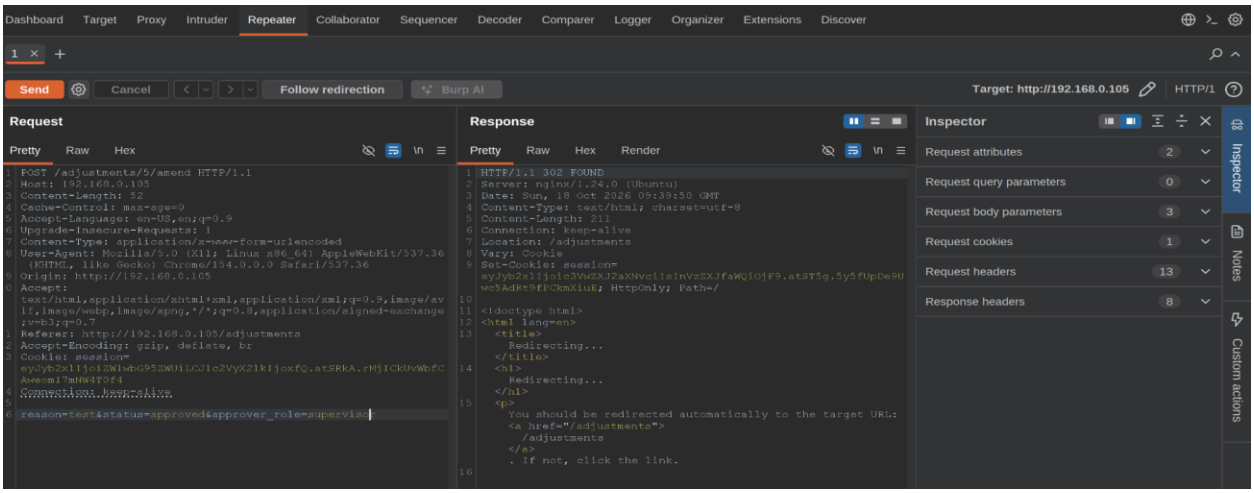
3. Add Reason : Overtime 10/24 and Hours 2. After doing this your requests looks like :



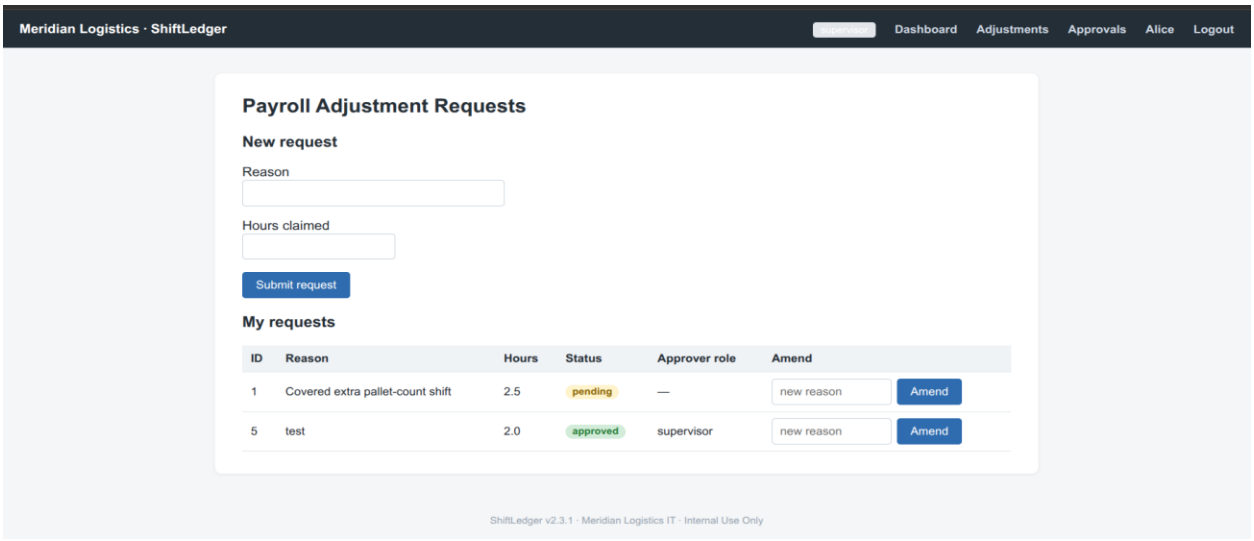
4.Click on Amend and Go to burpsuite —> Proxy —> Http history :

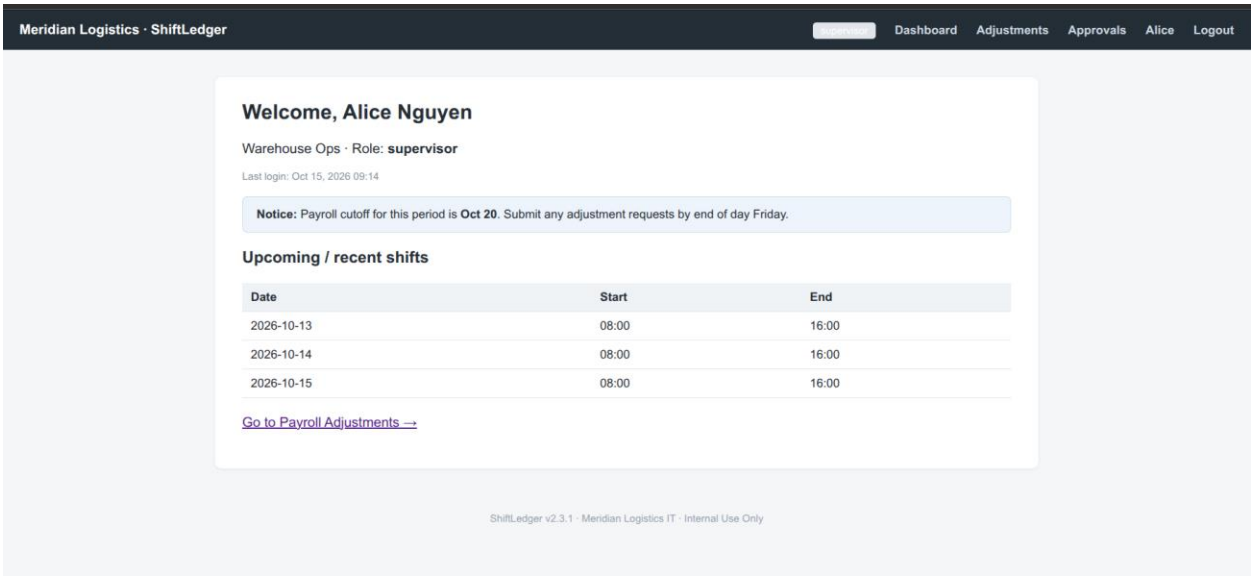


5. Send the POST /adjustment/5/amend request to repeater and add following code and send it :



6. Go to web page and reload your request from pending —> approved and see in upper tabs you role change from employee— > supervisor.





Impact:

- Unauthorized privilege escalation from employee to supervisor.
- Bypass of the intended approval workflow entirely.
- Enables all subsequent stages of this attack chain.

Remediation:

- Explicitly allow-list the fields an endpoint may update, server-side.
- Never derive authorization or role state from a field in the same request that modifies it.
- Separate 'edit own content' endpoints from any endpoint capable of changing approval/role state.

10. Stage 2 — Trusted-Field Role Escalation

Supervisor—> Finance_admin

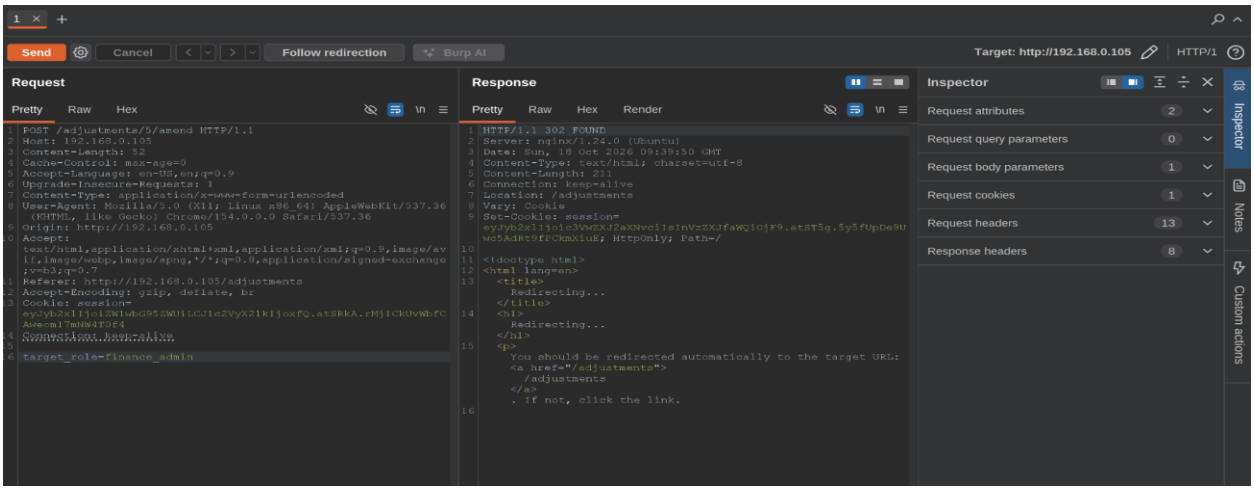
Severity	High
OWASP Category	A01:2021 – Broken Access Control
CWE	CWE-285 – Improper Authorization
Affected Endpoint	POST /supervisor/grant

Description:

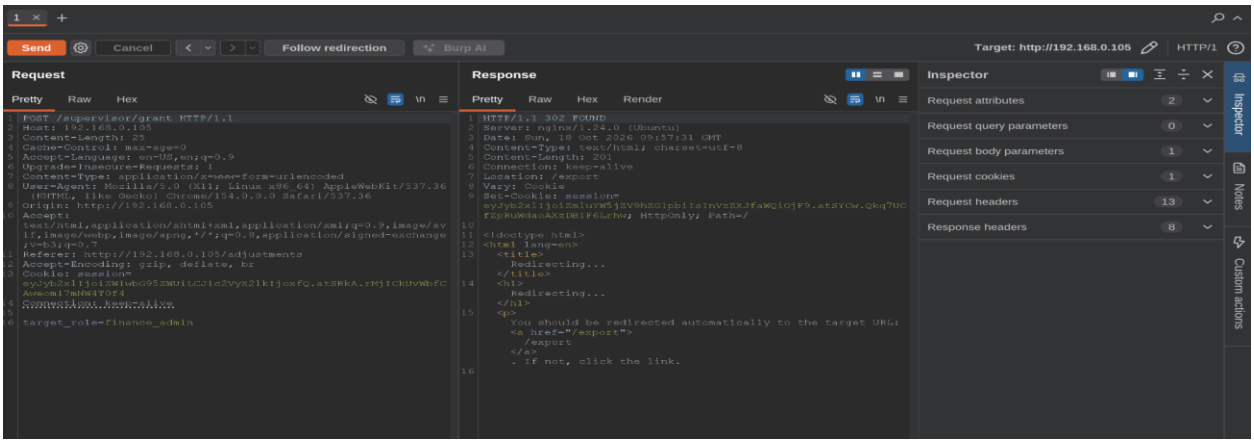
The /supervisor/grant endpoint is intended to let a supervisor delegate finance-export access for a specific task. It checks only that the caller already holds supervisor or finance_admin, then trusts a client-supplied target_role field for the elevation itself, rather than validating a specific, narrowly-scoped finance-delegation permission server-side.

Steps to reproduce :

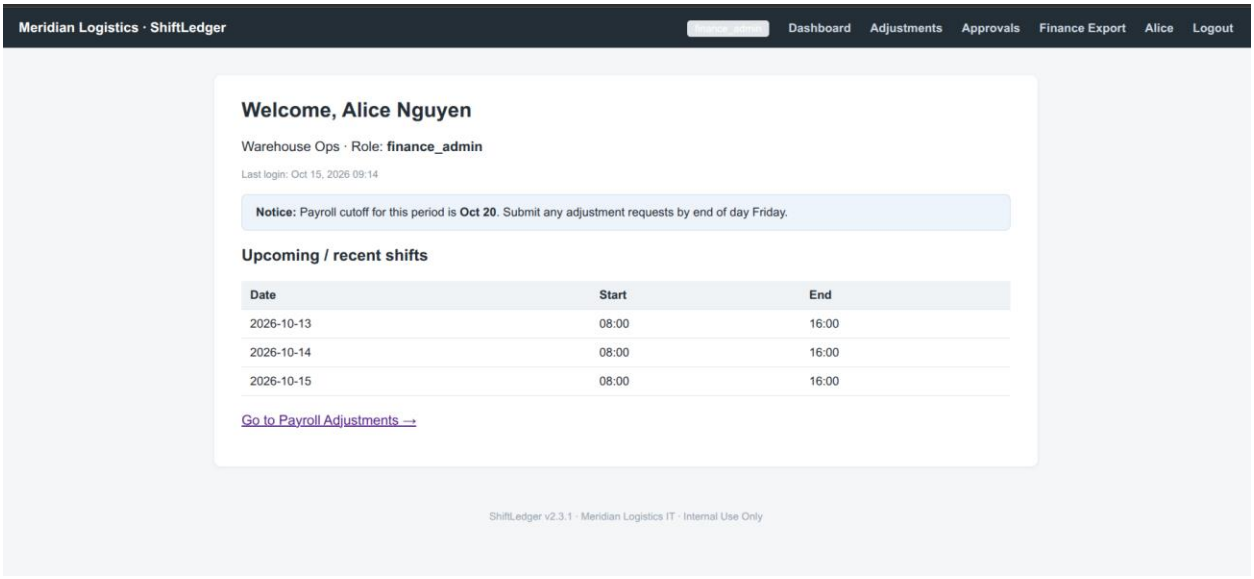
1. First go to the previous request and remove the reason,status and approver_role and add target_role=finance_admin



Second in this request change the request to /supervisor/grant and forward the request :



2. Go to web and see your role change to finance admin and you get finance-export access :



Impact:

- Unauthorized escalation to the highest-privileged application role.
- Grants access to the Finance Export feature exploited in Stage 3.

Remediation:

- Never trust a client-supplied value for the role being granted.
- Require a specific, auditable delegation permission rather than a coarse role check.
- Log and alert on all role-elevation events.

11. Stage 3 — Constrained Server-Side Template Injection:

Finance Admin → Code Execution (svc_shiftledger)

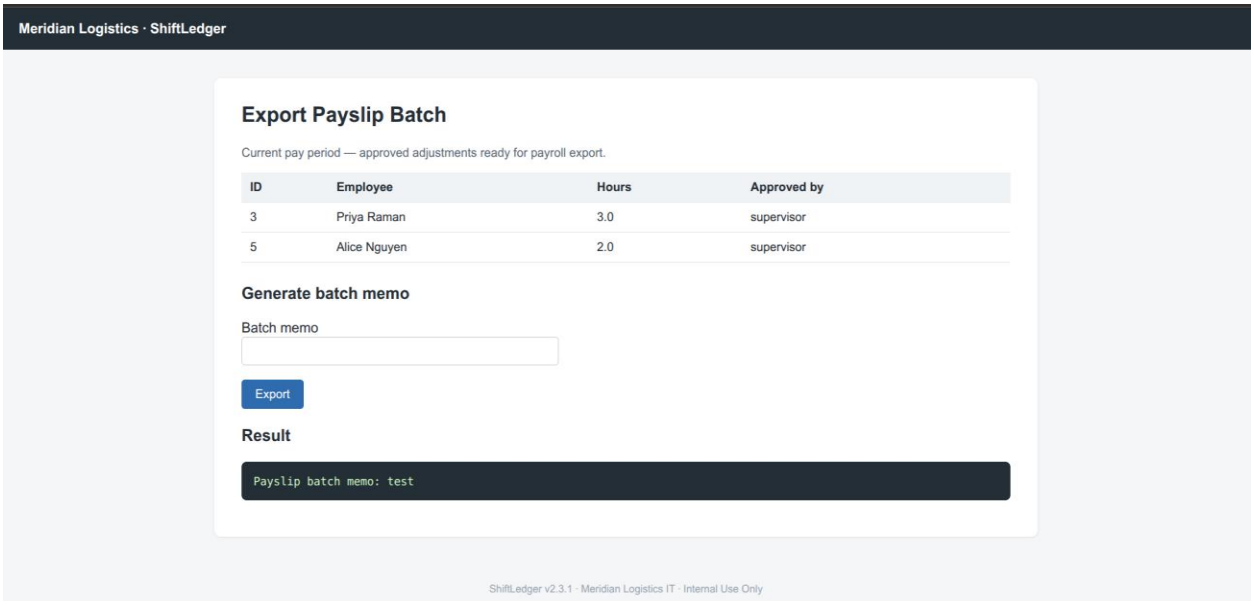
Severity	Critical
OWASP Category	A03:2021 – Injection
CWE	CWE-1336 – Improper Neutralization of Special Elements Used in a Template Engine
Affected Endpoint	POST /export

Description:

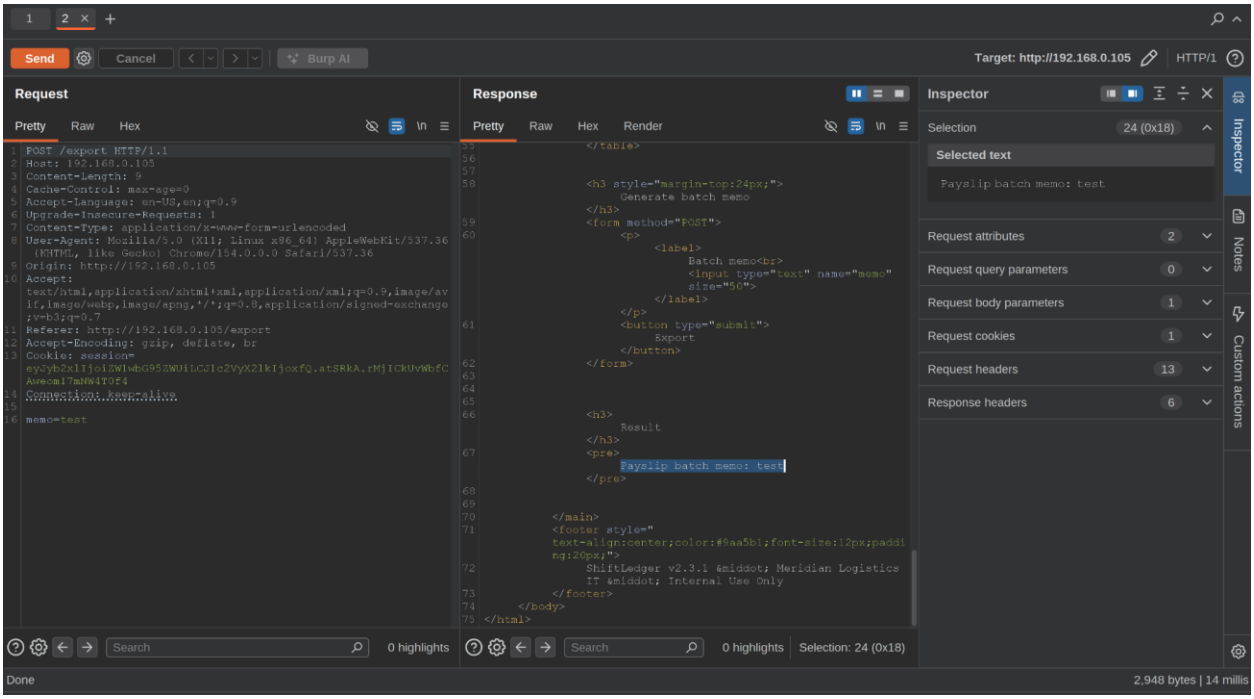
The Finance Export page's 'Batch memo' field is inserted directly into a Jinja2 template string server-side, rather than passed in as render context data, making it vulnerable to server-side template injection. A substring filter blocks the literal strings __, import, popen, system, exec(, and eval(, preventing the textbook payload and requiring a genuine bypass.

Steps to reproduce :

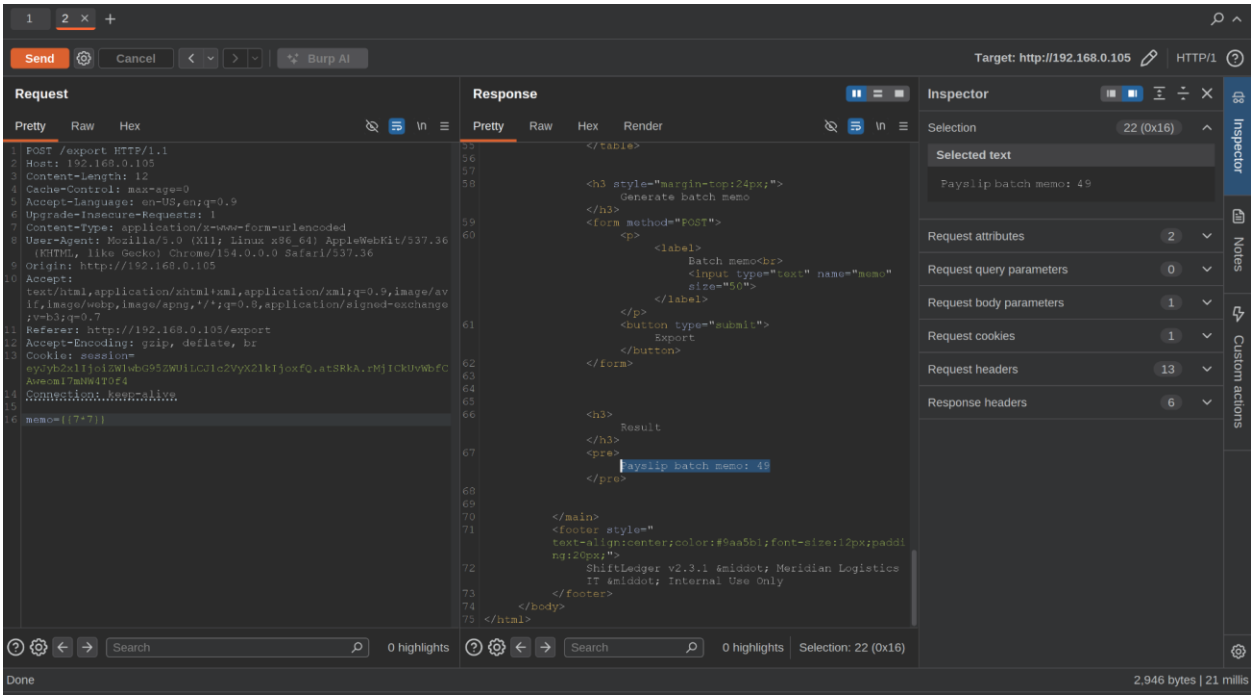
1. Go to finance export and add test in batch memo and export :



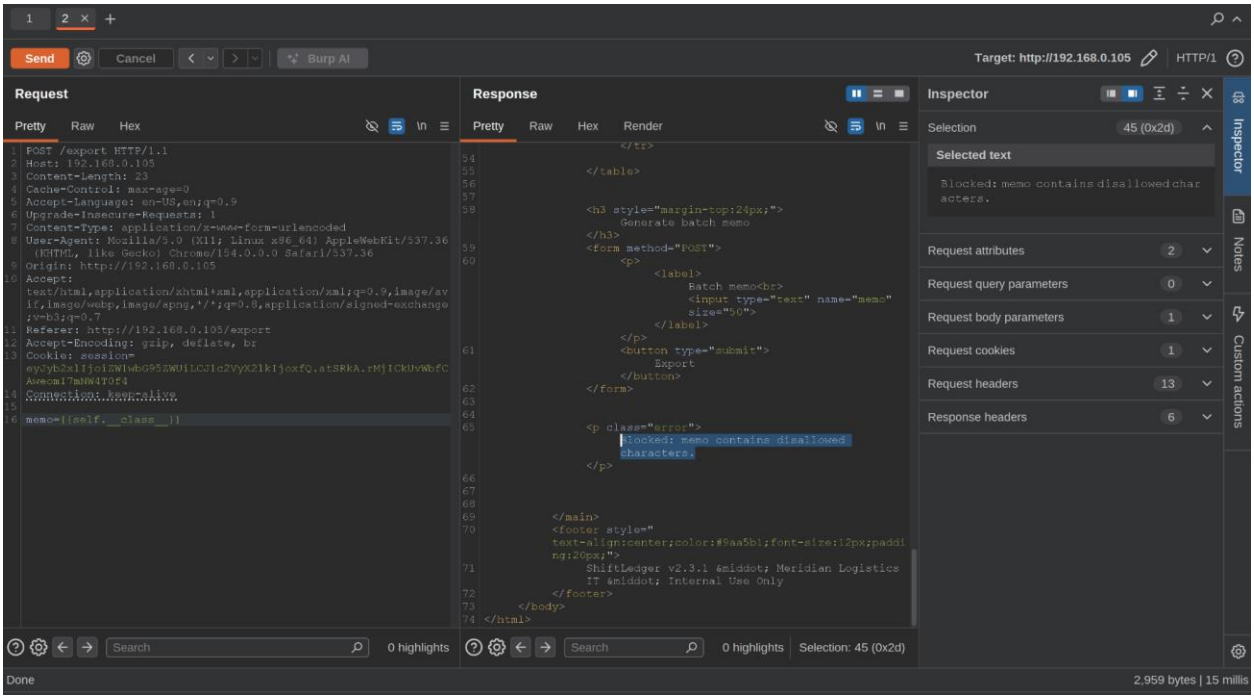
2. Go to burpsuite http history and see there is a POST /export request send it to repeater :



3. Add sample sink in the memo and see our sink renders 49 :

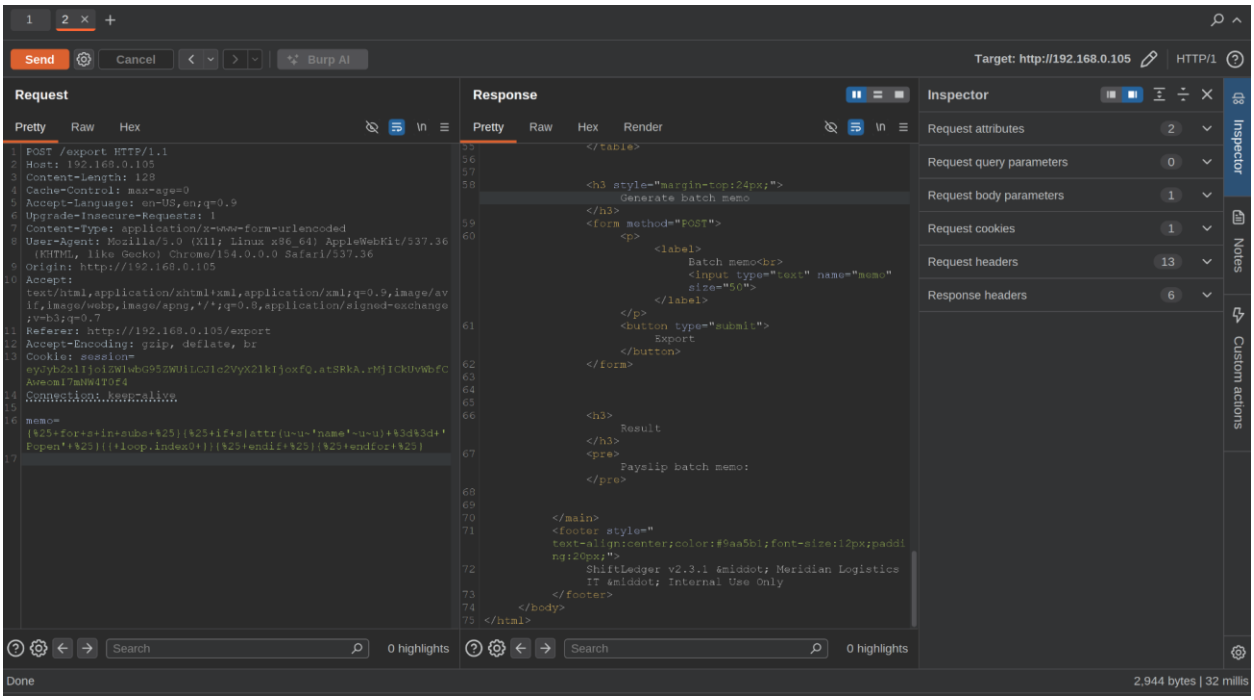


4. Add a native SSTI payload see what it render :

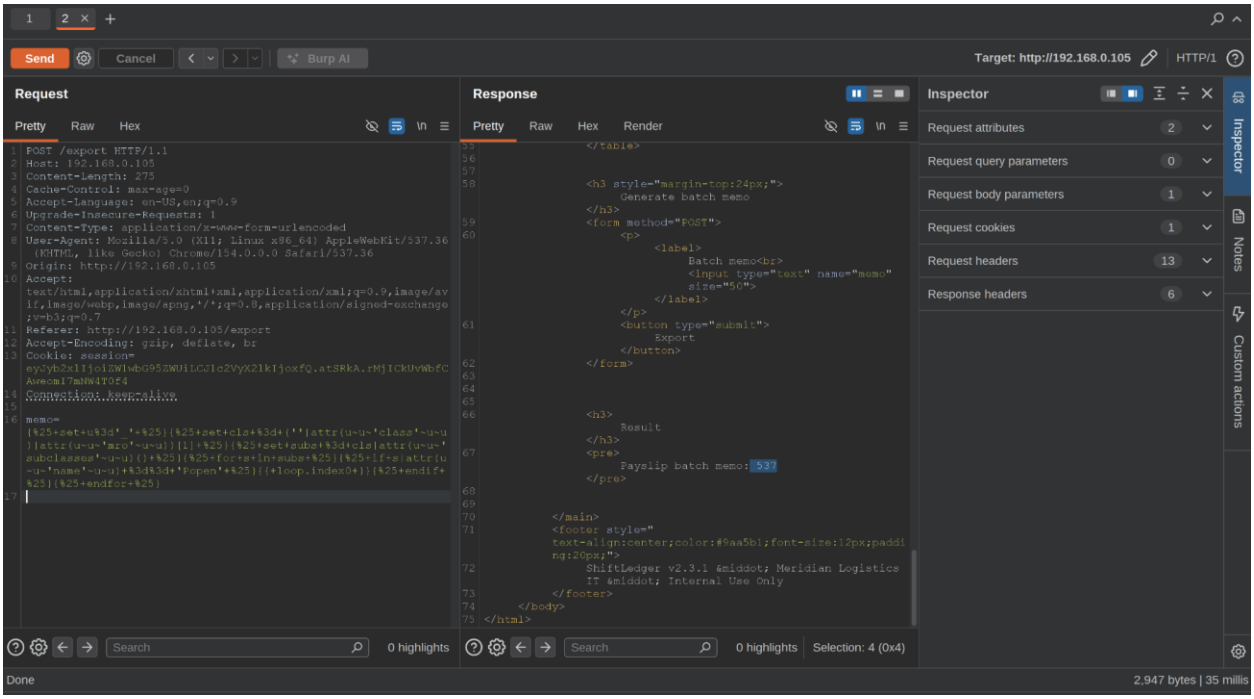


This confirms that our is blocked and confirm that a bypass is required .

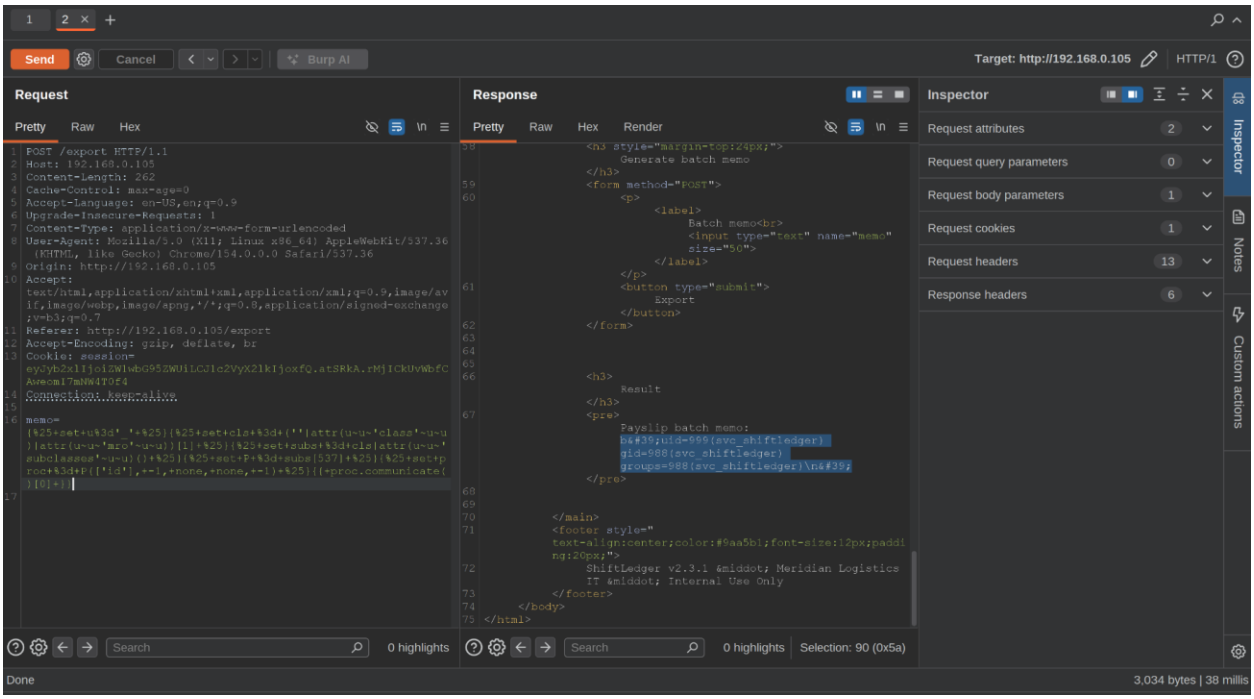
5. Use following to bypass blocked strings :



6. You can see we bypass the blocked strings. After that we want to know index to executes arbitrary commands. Use following payload :



7. Now we know the index so to confirm the arbitrary command execution as the application's unprivileged service account. Use following paylaod :



Impact:

- Arbitrary command execution on the target machine.
- Direct foothold enabling Stage 4 privilege escalation to root.

Remediation:

- Never insert user-supplied input into template source; pass it as render context data instead.
- Use a sandboxed template environment and avoid attribute-based object traversal.
- Apply defense-in-depth: a denylist filter alone is not a reliable control.

12. Stage 4 — Insecure Deserialization :

svc_shiftledger → Root

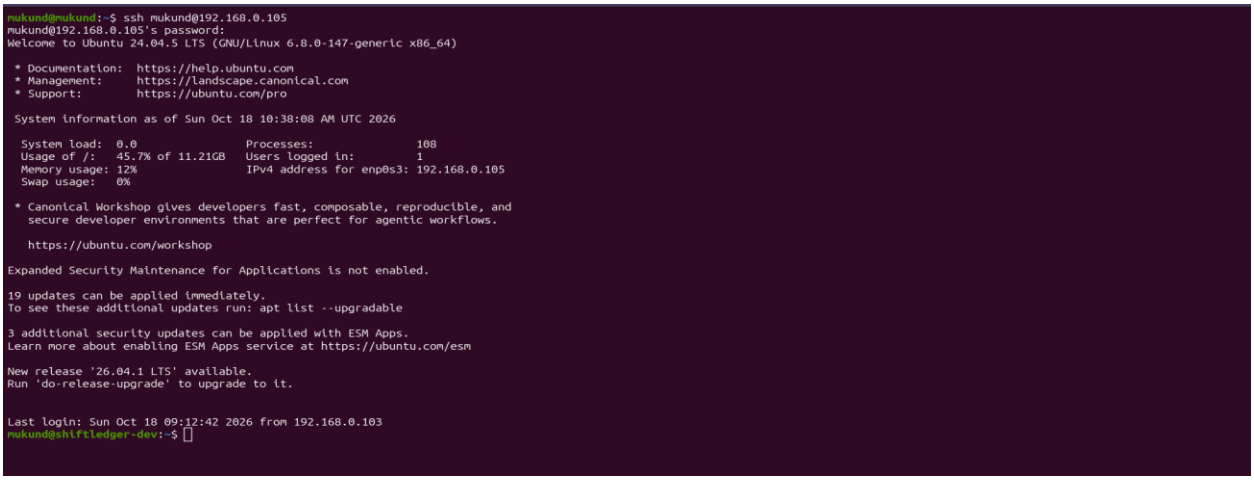
Severity	Critical
OWASP Category	A08:2021 – Software and Data Integrity Failures
CWE	CWE-502 – Deserialization of Untrusted Data
Affected Endpoint	shiftledger-backup.service (local, /var/shiftledger/jobs/)

Description:

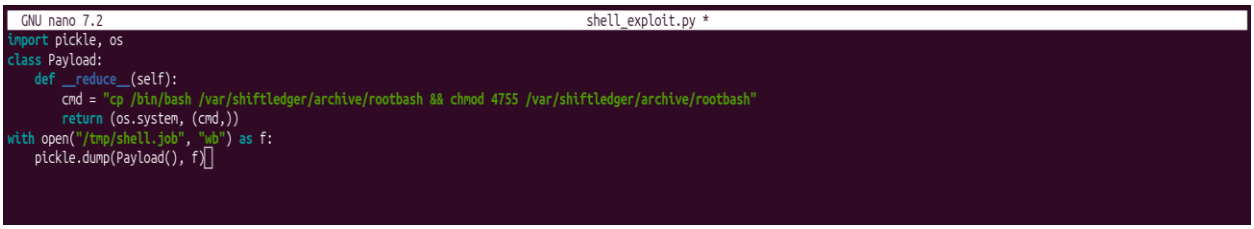
A separate backend service, shiftledger-backup, runs continuously as root via systemd. It polls /var/shiftledger/jobs/ (writable by the application's service account) for *.job files and deserializes each one with Python's pickle.load() rather than a safe format such as JSON. Because unpickling reconstructs objects by invoking whatever constructor or function they reference via __reduce__, a crafted file executes arbitrary code the instant the root-owned daemon loads it. Using the Stage 3 foothold, a malicious pickle file was delivered into the daemon's watched directory, chaining the entire attack through a single foothold.

Steps to reproduce :

1. Login into machine via ssh :



2. Create a file name as shell_exploit.py and use following payload and save it:



3. Execute this payload and go to /tmp folder :

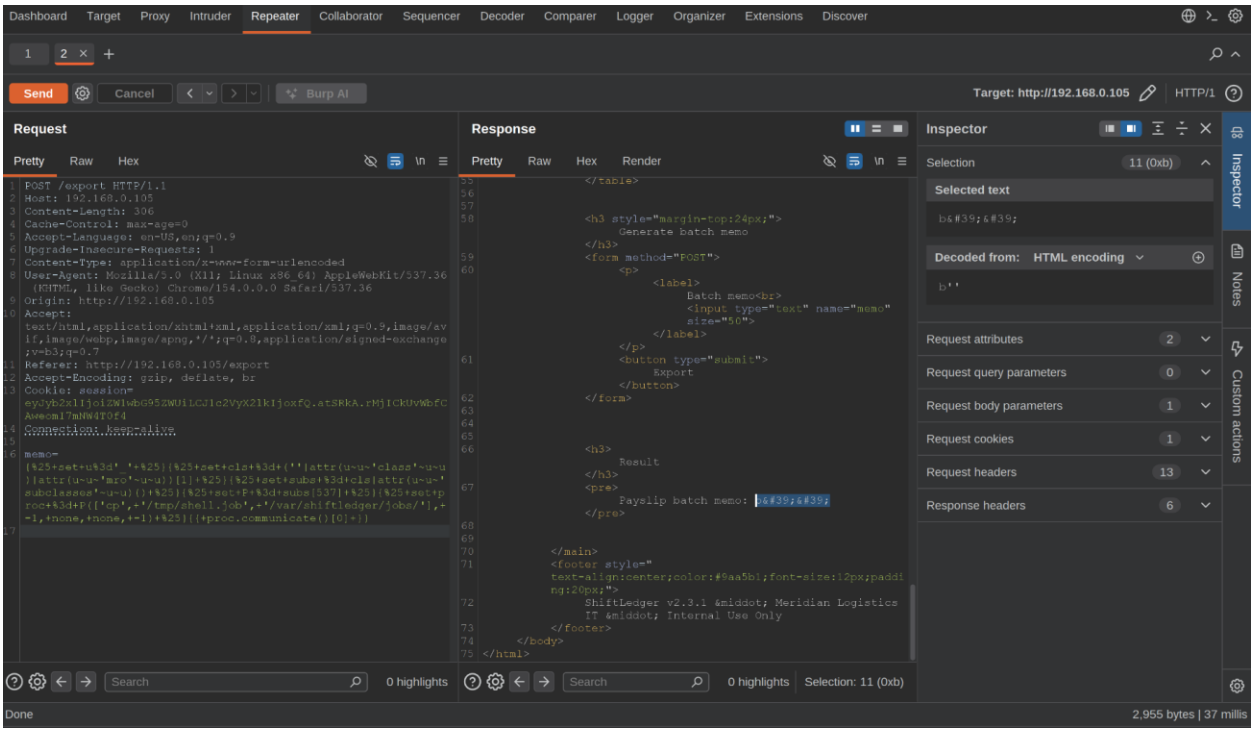
```
mukund@shiftledger-dev:~$ cd /tmp/
mukund@shiftledger-dev:/tmp$ ls
shell.job                                                    systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-systemd-logind.service-NnElOH
snap-private-tmp                                             systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-systemd-resolved.service-7pdD4K
systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-fwupd.service-OyIsGf   systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-systemd-timesyncd.service-MRuZgc
systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-ModemManager.service-YqVcnC  systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-upower.service-9vtc15
systemd-private-0bb1a1af2b0a432b9315fe8af52d37d3-polkit.service-3tfHx4
mukund@shiftledger-dev:/tmp$
```

4. You can see the shell.job file is there so give permissions to this file :

```
mukund@shiftledger-dev:/tmp$ chmod 644 /tmp/shell.job
mukund@shiftledger-dev:/tmp$ la -la shell.job
-rw-r--r-- 1 mukund mukund 132 Oct 18 10:44 shell.job
mukund@shiftledger-dev:/tmp$
```

5. Use the following payload and delivered to stage 3 SSTI RCE :

{%25+set+u%3d'_'+%25}{%25+set+cls+%3d+("|attr(u~u~'class'~u~u)|attr(u~u~'mro'~u~u))[1]+%25}{%25+set+subs+%3d+cls|attr(u~u~'subclasses'~u~u)()+%25}{%25+set+P+%3d+subs[537]+%25}{%25+set+proc+%3d+P(['cp',+'/tmp/shell.job',+'/var/shiftledger/jobs/'],+-1,+none,+none,+-1)+%25}{{+proc.communicate()[0]+}}



Within the daemon's ~2 second poll interval, the job is picked up and deserialized as root, creating a SUID root copy of bash. Launching it with -p (to preserve privileges) yields an interactive root shell:

```
mukund@shiftledger-dev:/tmp$ /var/shiftledger/archive/rootbash -p
rootbash-5.2#
rootbash-5.2# whoami
root
rootbash-5.2# id
uid=1000(mukund) gid=1000(mukund) euid=0(root) groups=1000(mukund),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),101(lxd)
rootbash-5.2#
```

Impact:

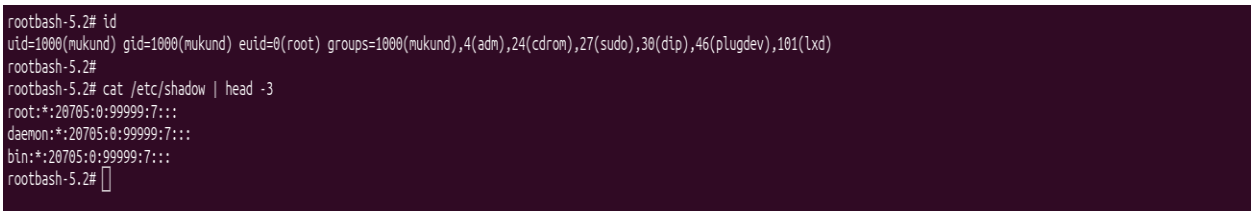
- Full compromise of the target machine.
- Complete loss of confidentiality, integrity, and availability.

Remediation:

- Never deserialize untrusted data with pickle; use JSON or another safe format.
- Run background services with the minimum privilege required, not root.
- Validate and restrict which accounts may write to a privileged service's input directory.

13. Final Proof of Compromise :

Starting from a single low-privilege employee login, the four chained stages above yield a fully interactive root shell on the target machine, confirmed via id showing euid=0(root).



14. Recommendations:

Finding	Recommendation
Mass Assignment	Allow-list updatable fields server-side; never trust client-supplied role/status fields.
Trusted-Field Escalation	Require specific, auditable delegation permissions rather than coarse role checks.
Server-Side Template Injection	Never render user input as template source; use a sandboxed engine.
Insecure Deserialization	Replace pickle with JSON; run background services with least privilege.

15. Conclusion :

This assessment demonstrates a complete, four-stage attack chain against ShiftLedger, escalating from a single low-privilege employee account to full root compromise of the target machine. Each stage required genuine analysis — discovering an unintended field, identifying a trust boundary failure, constructing a working filter bypass, and exploiting an unsafe deserialization pattern — rather than relying on default credentials, a single public exploit, or an automated scanning tool.

16. References :

1. OWASP Foundation. OWASP Top 10 (2021). https://owasp.org/www-project-top-ten/

2. OWASP Foundation. OWASP Web Security Testing Guide (WSTG). https://owasp.org/www-project-web-security-testing-guide/

3. PortSwigger Ltd. Server-Side Template Injection — background reading (general concept, no labs or payloads reused). https://portswigger.net/web-security

4. MITRE Corporation. Common Weakness Enumeration (CWE). https://cwe.mitre.org/

5. Python Software Foundation. pickle — Python object serialization (documentation). https://docs.python.org/3/library/pickle.html

6. PortSwigger Ltd. Burp Suite Documentation. https://portswigger.net/burp/documentation